

02 · Scheduling-Theorie

Task-Modelle, Utilization, RMS/EDF · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 66

Willkommen zurück zu Echtzeitsystemen!

- Herzlich willkommen zum 2. Termin
- Letzte Woche: Rechtzeitigkeit, Cortex-M4, Interrupts, Bare-Metal
- Heute: Task-Modell, Auslastung, RMS, RTA, EDF — **ein Prozessor**
- Praxis: kooperativer Round-Robin als Dispatcher (kein Register-Port)
- Der zweite Kern des H755 verdoppelt dieses Modell nicht automatisch

Notizen

Zeiten und Begriffe wie in VL1: Release, Start, Completion, relative/absolute Deadline, Jitter. Alle Kernbeispiele gelten für genau eine CPU. Bus- und Speicherinterferenz des H755 liegen außerhalb des idealisierten Lehrmodells, sofern nicht im WCET enthalten.

Folie

2 / 66

Agenda für heute

1. Recap und Task-Modell (C , T , D , R)
2. Auslastung, WCET, Scheduling-Klassen
3. RMS-Schranke, Gantt, Response-Time Analysis
4. EDF am selben Zahlenbeispiel, Überlast
5. SysTick/PendSV und Labor-Dispatcher

Notizen

Durchgängige Tasksets: 75 % (Präemption), 85 % (RTA oberhalb der Schranke), 97 % (RMS scheitert, EDF hält).

Lernziele des heutigen Tages

1. Task, Job und C_i, T_i, D_i, R_i auf einer Zeitachse erklären
2. Auslastung U berechnen und von lokalen Fristen unterscheiden
3. Kooperativ/präemptiv und Fixed Priority/EDF als getrennte Achsen
4. Eine einfache RTA selbst rechnen und die EDF-Annahmen prüfen
5. Den Labor-Dispatcher ohne rekursive Tasks erklären

Notizen

Am Ende: $R \leq D$ unter benannten Annahmen nachweisen, Liu-Layland von einem exakten Test unterscheiden, und den Callback-Scheduler vom RTOS-Thread trennen.

Recap: Polling vs. Interrupts

- Begrenzte Super-Loop kann Fristen halten
- Lange unzerlegte Arbeit sprengt das Budget
- Interrupts verkürzen bestimmte Reaktionen
- Maskierung, ISR-Zeit und Ressourcen bleiben

```
while (1) {  
    Sensor_Auswerten(); /* z. B. 50 ms */  
    LED_Blinken();      /* verpasst das Budget */  
}
```

Notizen

Polling ist nicht grundsätzlich ungeeignet. Interrupts stellen keinen automatischen Determinismus her — Anschluss an VL1.

Recap: Die Spaghetti-Code-Falle

- Viele Ereignisse: Flags in der Main-Loop
- Prioritäten und Fristen werden schwer nachvollziehbar
- Ein RTOS strukturiert Tasks — es beweist Fristen nicht von selbst

Notizen

Flag-Suppe ist ein Strukturproblem. Der spätere Scheduler arbeitet mit Zuständen und Prioritäten, nicht mit einem automatischen WCET-Wächter.

Der Paradigmenwechsel: Das Betriebssystem

- Zeitmanagement kann ein RTOS übernehmen
- RTOS-Thread: eigener Stack, Kontextwechsel
- Labor heute: endliche Schritte, Rückkehr zum Dispatcher
- Das sind unterschiedliche Kontextmodelle

Notizen

Ein Job ist nur bei passend strukturierter Anwendung ein Schleifendurchlauf. FreeRTOS-Threads und die Labor-Callbacks nicht vermischen.

Was ist Scheduling?

- Ablaufplanung: wer erhält die CPU wann?
- Heute: genau **ein** Prozessor
- Echtzeit: spezifizierte Fristen, nicht pauschale Fairness
- Fairness und Deadline-Einhaltung sind verschiedene Ziele

Notizen

Frage: Warum darf eine zweite H755-CPU die heutige Formel $U \leq 1$ nicht einfach verdoppeln? Weil das Modell, Busse und der Nachweis sich ändern.

Vom C-Code zum Modell

- Analyse beschreibt den Entwurf: C, T, D
- Ein Fixed-Priority-Scheduler zur Laufzeit kennt typisch Zustand und Priorität
- Er leitet daraus keine Deadline-Garantie ab
- FreeRTOS überwacht WCET/Deadline nicht automatisch

Analyse	Laufzeit (Fixed Priority)
C, T, D , Nachweis $R \leq D$	Ready/Running/Blocked, konfigurierte Priorität

Notizen

Der mathematische Fahrplan und der RTOS-Scheduler sind zwei Schichten. Garantien gelten unter den Annahmen der Analyse, nicht für jedes STM32-Programm.

Definition: Was ist eine Task?

- Task τ_i : wiederkehrende Aufgabe
- Job $J_{i,k}$: eine konkrete Aktivierung
- RTOS-Thread: eigener Kontext (Register, Stack)
- Labor-Callback: endlicher Schritt, Zustand im TCB, kein pausierter Stack

Notizen

Task $\neq$ Job. Ein `while (1)`-Durchlauf ist nur dann ein Job, wenn Release und Ende so definiert sind.

Parameter einer Task: C_i (Ausführungszeit)

- C_i : eigener CPU-Bedarf ohne Unterbrechungszeit
- Im harten Analysemodell: angenommene WCET
- Warte- und Präemptionszeit zählen nicht zu C_i
- Ohne dokumentierte Schranke kein Nachweis

Notizen

Messmaximum plus beliebiger Puffer ist kein WCET-Beweis. Anschluss an VL1.

Folie

11 / 66

Parameter einer Task: T_i (Periode)

- T_i : Abstand der **Releases**, nicht der CPU-Starts
- Beispiel Regler: $T_i = 10$ ms (Lehrannahme)
- Zunächst: kein Release-Jitter

Notizen

Start kann später liegen als Release. T und die relative Deadline D nicht gleichsetzen, außer das Modell sagt $D = T$.

Folie

12 / 66

Parameter einer Task: D_i (Deadline)

- Relative Deadline D_i : Frist ab Release
- Absolute Deadline: $d_{i,k} = r_{i,k} + D_i$
- Job-Antwortzeit: $R_{i,k} = f_{i,k} - r_{i,k}$
- R_i : nachgewiesene Worst-Case-Antwortzeit; gefordert $R_i \leq D_i$
- Completion zum Zeitpunkt d gilt als rechtzeitig

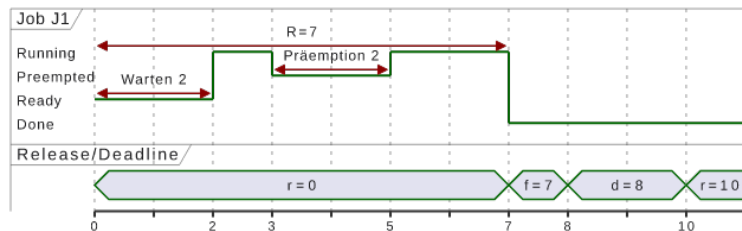
Notizen

R enthält Warten und Präemption. $C \leq D$ ist nötig, aber bei mehreren Tasks nicht hinreichend.

Das grafische Task-Modell

Lehrjob (ms): $r = 0$, Start $s = 2$, Ausführung 2–3 und 5–7, Präemption 3–5, $f = 7$, $D = 8$, $T = 10$.

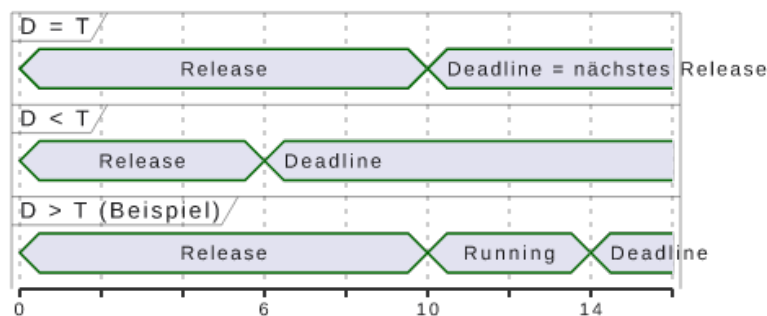
- $C = 3$, Startwartezeit 2, Unterbrechung 2, $R = 7 \leq D$
- Nächster Release bei $t = 10$, nicht beim CPU-Start



Notizen

Beide Laufstücke gehören demselben Job. Im Lehrmodell läuft er mit der angenommenen WCET. Frage: Warum ist $T = 10$ nicht der Abstand der Starts?

Deadline-Klassifizierung



- Implicit: $D = T$; constrained: $D \leq T$ (enthält implicit)
- Arbitrary: D beliebig, auch $D > T$ und überlappende Jobs — nicht „immer $D > T$ “
- Liu-Layland und der einfache EDF-Test heute: $D = T$

Notizen

Die dritte Zeile ist ein Beispiel für arbitrary deadlines, keine Definition. Frage: Warum reicht $U \leq 1$ bei $D < T$ nicht?

Task-Typen nach Auslöser

- Periodisch: feste Release-Abstände T
- Sporadisch: garantierter Mindestabstand, nicht „zufällig“
- Aperiodischer Burst: ohne Lastgrenze keine pauschale Garantie für jeden Eingang
- Begrenzen: Zulassung, Verwerfen, reserviertes Budget (Server-Theorie später)

Gegenprobe, synchron, eine CPU: zwei Tasks $C = 2\text{ ms}$, $T = 10\text{ ms}$, $D = 3\text{ ms}$.

- $U = 40\%$, aber 4 ms Arbeit vor $t = 3\text{ ms}$ — unabhängig vom Algorithmus unmöglich

Notizen

Ein Taster ohne Entprellung hat keine garantierte Mindestreleasezeit. Physikalische Betätigung und modellierte Mindestankunft trennen. Die 40-%-Gegenprobe zeigt, warum $U \leq 1$ bei kurzen Deadlines kein EDF-Test ist.

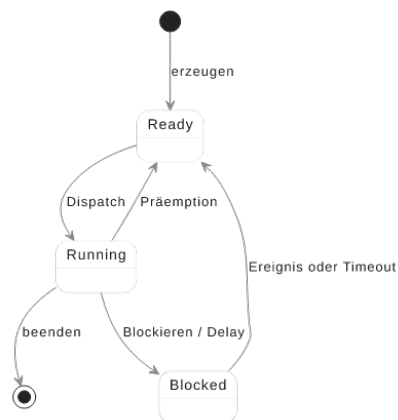
Das RTOS Task-Zustandsmodell

- Running: hat die CPU (eine Nutz-Task; Idle wenn keine Arbeit)
- Ready: will rechnen
- Blocked: wartet auf Ereignis oder Timeout, rechnet nicht
- Blocked wird Ready; die Auswahl kann unmittelbar folgen
- Interruptkontext ist kein vierter Task-Zustand

Notizen

Auf einer CPU höchstens ein Task-Kontext Running. ISRs separat. Idle darf laufen, wenn niemand Ready ist.

Visualisierung der Zustandsmaschine



- Scheduler wählt, Dispatcher führt aus
- RTOS: Kontextwechsel; Labor: Funktionsaufruf und Rückkehr

Notizen

Ein geweckter Job wird nicht „magisch Running“, sondern zuerst Ready.

Metrik: CPU-Auslastung (U)

$$U = \sum_{i=1}^n \frac{C_i}{T_i}$$

- Langfristiger Pflichtanteil auf **einer** CPU
- $U > 1$: dauerhafte Pflichtlast nicht erfüllbar
- Endliche Bursts und eine zweite H755-CPU sind ein anderes Modell
- Nicht zugewiesene Prozent sind kein Fristpuffer in jedem Fenster

Notizen

U ist notwendig für unbegrenzte periodische Pflicht, nicht hinreichend bei $D < T$.

Auslastung: Ein Rechenbeispiel

Durchgängiges 85%-Set (ms), $D_i = T_i$:

Task	C	T	U_i
τ_1	1	4	25 %
τ_2	2	5	40 %
τ_3	2	10	20 %

- $U = 0,85$; restliche 15 %: nicht zugewiesen im Langfristmodell
- Lokal kann die Nachfrage trotzdem eng sein (RTA gleich)
- ISR-, Dispatcher- und Messoverhead getrennt bilanzieren, nicht doppelt und nicht als „Pauschalreserve = Beweis“

Notizen

Die 15 % sind keine Garantie, dass jeder Job 15 % Reserve hat. Overhead nur einrechnen, wenn er nicht schon in C steckt.

Exkurs: Das Problem mit der WCET

```
if (sensor_value > 1000)
    calculate_fft(); /* illustrative Laufzeit, z. B. länger */
else
    flag = 1;        /* kurzer Pfad */
```

- Eine Mittelwertannahme von 1 ms braucht eine Pfadverteilung oder bleibt hypothetisch
- Beobachtetes Maximum $\neq$ sichere Obergrenze
- Statische Analyse braucht Plattform- und Codeannahmen (Cache, Bus, Waitstates)

Notizen

Lehrkurve, keine Boardmessung. H755-Interferenz gehört ins WCET-Modell oder wird als außerhalb des idealen Ein-CPU-Modells gekennzeichnet.

Wie ermittelt man die WCET?

Statische Analyse

- Pfade, Schleifengrenzen, Plattform
- Sichere Schranke nur unter Annahmen

Messung

- Messmaximum der Läufe
- Kein Beweis durch „plus Puffer“
- Annahmen dokumentieren

Notizen

Beide Wege brauchen ein Modell. „Messen und 20 % drauf“ ist eine Ingenieurpraxis, kein Nachweis.

Zusammenfassung Task-Modell

- Task vs. Job; T = Release-Abstand
- C eigener Bedarf; $R = f - r$; Nachweis $R \leq D$
- U auf einer CPU; $U > 1$ schließt dauerhafte Pflichtlast aus
- Analysezahlen $\neq$ automatische RTOS-Überwachung

Notizen

Frage: Erfüllt $U = 40\%$ mit $D = 3$ ms und zwei Jobs à 2 ms die Frist? Nein.

Die Kernfrage des Scheduling

- Mehrere Ready, eine CPU
- Der Algorithmus legt die Auswahlregel fest
- Ob Fristen halten, zeigt der Test unter seinen Annahmen

Notizen

Auswahlregel und Beweis nicht vermischen.

Scheduler vs. Dispatcher

- Scheduler: wer als Nächstes (Priorität, Deadline, Ring)
- Dispatcher: setzt die Auswahl um
- RTOS-Thread: Register/PSP; Labor: `step()` aufrufen und Rückkehr abwarten

Notizen

Im Labor gibt es keinen PendSV-Registerwechsel.

Klassifizierung: Offline vs. Online

Achse	Ausprägungen
Zeitpunkt der Planung	offline (Tabelle) / online (zur Laufzeit)
Auslöser	zeitgesteuert / ereignisgesteuert
CPU-Abgabe	kooperativ / präemptiv
Auswahl	Fixed Priority / EDF / RR

- Die Achsen nicht gleichsetzen: online $\neq$ automatisch ereignisgesteuert

Notizen

Eine offline Tabelle kann zeitgesteuert sein. Ein Fixed-Priority-RTOS kann auf Ereignisse reagieren, nicht nur auf den Tick.

Klassifizierung: Kooperativ vs. Präemptiv

- Kooperativ: Task gibt die CPU frei (`yield` / Rückkehr)
- Präemptiv: der Kern entzieht die CPU
- Fixed Priority ist nicht automatisch RMS: Prioritäten muss die Anwendung passend setzen
- Round-Robin ignoriert Fristen, kann sie aber unter nachgewiesenen Bedingungen erfüllen

Notizen

Prioritätswahl und Art der CPU-Abgabe sind verschiedene Entscheidungen.

Kooperatives Scheduling (Non-preemptive)

- Ein Schritt, der nie zurückkehrt, hält den Dispatcher an
- Aktive IRQs können weiterlaufen
- `Calculate_PID` und `Apply_PWM` im Beispiel als **ein** begrenzter Schritt, wenn die Aktorfrist das verlangt
- `yield` dazwischen nicht unkommentiert einbauen

Notizen

Kooperativ kann Fristen erfüllen, wenn jeder Schritt ein nachgewiesenes Budget hat und zurückkehrt.

Präemptives Scheduling

- Eine ISR kann eine höherpriorie Task Ready machen
- PendSV kann **nach ISR-Ende** wechseln — nicht erst beim nächsten Tick
- Präemption verhindert keinen Starvation, wenn High ständig Ready bleibt
- Shared Data: später Mutex; Shared Memory ist nicht pauschal verboten, Abhängigkeiten müssen im Modell stehen

Notizen

„Alle Wechsel passieren am Tick“ ist falsch für ereignisgesteuerte Wechsel. Frage: Warum läuft Low nicht, wenn High nur `yield` macht, aber Ready bleibt?

Prioritätsbasiertes Scheduling

- Fixed Priority: konfigurierte Priorität, statisch im RMS-Modell
- EDF: unter bereiten Jobs die früheste **absolute** Job-Deadline
- Eine Deadline wird nicht allein dadurch „höher“, dass Zeit verstreicht, während andere absolute Deadlines fest bleiben
- NVIC-IRQ: kleinere Zahl = dringender; native FreeRTOS-Taskpriorität: größere Zahl = höher (CMSIS-RTOS2 separat prüfen)

Notizen

IRQ-Priorität und Task-Priorität nicht vermischen.

Folie

30 / 66

Der Spezialfall: Round-Robin (RR)

- Gleiche Auswahlklasse, ringförmige Zeitscheiben **oder** kooperative Schritte
- Timer-RR: gleiche Quanten; Labor-RR: keine erzwungenen 1-ms-Quanten
- Fairness ersetzt keinen Fristnachweis

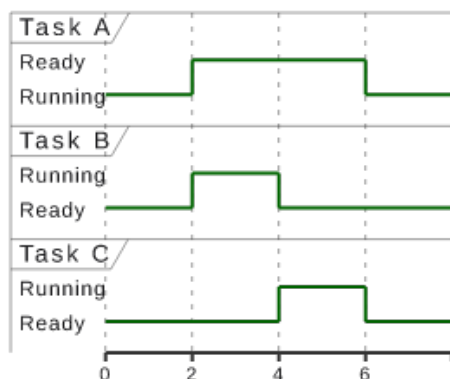
Notizen

Zwei Bilder im Laborteil: gleiche Scheiben vs. unterschiedlich lange Schritte.

Folie

31 / 66

Visualisierung: Round Robin



- Genau eine Running-Spur; idealisierte gleiche Scheiben

Notizen

Das ist timerpräemptives RR, nicht der Labor-Dispatcher.

Das Problem des Aushungerns (Starvation)

- Strikte Priorität, High bleibt Ready (auch nach `yield`): Low kommt nicht dran
- `yield` behebt das nicht, solange High sofort wieder gewählt wird
- Abhilfe: Blockieren, periodische Freigabe, geprüfte Budgets
- Kooperativ zusätzlich: ein Schritt ohne Rückkehr stoppt die anderen Callbacks

Notizen

Präemption garantiert weder Fairness noch Jitterfreiheit.

Wann ist ein System schedulbar?

- **Feasibility:** es existiert irgendein zulässiger Plan
- **Schedulability:** ein **gewählter** Algorithmus hält die Fristen
- Ein RMS-Test kann RMS verwerfen, obwohl EDF funktioniert (97%-Beispiel)
- Tests gelten unter ihren Annahmen, nicht für jedes STM32-Programm

Notizen

Die 40%-Gegenprobe ist infeasible, nicht nur „RMS-ungeeignet“.

Schedulability-Tests

- Hinreichend: bestanden $\Rightarrow$ nachgewiesen; nicht bestanden $\Rightarrow$ **keine Entscheidung**
- Exakt im benannten Modell: bestanden oder widerlegt
- Die einfache RTA ist dieser Fixpunkt-Test, nicht pauschal „NP-schwer“
- Allgemeine Scheduling-Probleme können schwer sein — das hier benannte Verfahren nicht damit abtun

Notizen

Liu-Layland ist hinreichend. Darüber RTA, nicht sofort „unschedulable“.

Rate Monotonic Scheduling (RMS)

- Fixed Priority, präemptiv, Rate: kürzeres $T \Rightarrow$ höhere Priorität
- Liu & Layland (1973), für das ideale Ein-CPU-Modell
- Ein FP-RTOS ist nicht automatisch RMS, solange die Prioritäten nicht so vergeben sind

Notizen

Keine Branchenzuordnung „Bremse = RMS“. RMS ist eine Prioritätsregel plus Analyse.

RMS: Die mathematischen Annahmen

Für die Kernbeispiele, sofern nicht anders gesagt:

1. Ein Prozessor, vollständig präemptierbar, unabhängig
2. Beschränkte C_i , kein Blocking, kein Release-Jitter, keine Selbstsuspendierung
3. Zunächst Overhead/ISRs = 0 (sonst in C oder als Interferenz modellieren)
4. Liu-Layland und einfacher EDF-Test: $D_i = T_i$
5. Einfache RTA: Fixed Priority mit $D_i \leq T_i$
 - Shared Memory ist erlaubt, wenn Abhängigkeiten und Interferenz im Modell stehen
 - H755-Businterferenz: im WCET oder außerhalb dieses Idealmodells

Notizen

„Kein Shared Memory“ wäre zu hart. Entscheidend ist, ob die Störung in der Analyse vorkommt.

Beispiel 1: RMS-Prioritäten vergeben

75%-Set (ms), $D = T$, Release aller ersten Jobs bei $t = 0$:

- $\tau_1 = (1, 4)$, $\tau_2 = (2, 6)$, $\tau_3 = (2, 12)$
- $U = \frac{1}{4} + \frac{2}{6} + \frac{2}{12} = 0,75$
- Priorität $\tau_1 > \tau_2 > \tau_3$

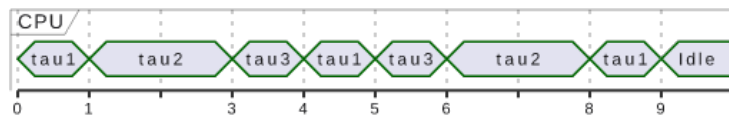
Notizen

Dieses Set zeigt Präemption von τ_3 bei $t = 4$.

Visualisierung: Präemptives RMS

Intervalle halboffen $[Start, Ende)$. CPU: 0–1 τ_1 , 1–3 τ_2 , 3–4 τ_3 , 4–5 τ_1 , 5–6 τ_3 , 6–8 τ_2 , 8–9 τ_1 , 9–12 Idle.

- τ_3 : $C = 2$, $R = 6 \leq 12$; Completion bei $t = 6$ hält
- Neues τ_2 startet bei seinem Release $t = 6$



Notizen

Bei $t = 4$ unterbricht τ_1 den noch offenen ersten Job von τ_3 . Critical Instant ist der gemeinsame **Release**, nicht ein gemeinsamer CPU-Start — auf einer CPU starten sie nicht gleichzeitig.

Der kritische Moment (Critical Instant)

- Für dieses FP-Modell: Release zusammen mit allen höherprioren Aufgaben
- „Hält hier, hält im Modell“ — nicht für Jitter, Blocking oder Selbstsuspendierung
- Angepasste Analysen, wenn die Annahmen fallen

Notizen

RTA für 75 %: $R_1 = 1$, $R_2 = 3$, $R_3 = 6$.

Die RMS-Schranke (Liu & Layland, 1973)

$$U \leq U_{LL}(n) = n(2^{1/n} - 1)$$

n	U_{LL}
1	100 %
2	82,84 %
3	77,98 %
4	75,68 %
∞	$\ln 2 \approx 69,31 \%$

- Hinreichend unter den LL-Annahmen, nicht notwendig
- 69,3 % ist **keine** allgemeine RMS-Obergrenze

Notizen

Harmonische Gegenprobe (Anhang in den Notizen): $(C = 2, T = D = 4)$ und $(C = 4, T = D = 8)$, $U = 100 \%$. Ablauf 0–2 τ_1 , 2–4 τ_2 , 4–6 τ_1 , 6–8 τ_2 hält unter RMS.

Werte der RMS-Schranke

Für $n = 3$:

- bis 77,98 %: unter den Modellannahmen durch die Schranke nachgewiesen
- darüber bis 100 %: genauer testen (RTA)
- über 100 %: dauerhafte Pflichtlast auf einer CPU nicht erfüllbar
- 75%-Set liegt im ersten Bereich; 85%-Set braucht RTA
- „Schranke nicht erfüllt“ $\neq$ „nicht schedulbar“

Notizen

Tabelle und Aussage bleiben auf einer Folie. Faustregel $\ln 2$ ist der Grenzwert, kein Verbot höherer Auslastung.

Interpretation der RMS-Schranke

- Die Schranke ist eine Entscheidungshilfe, kein vollständiger RMS-Test
- FP-Prioritäten im RTOS müssen zur Rate passen, sonst ist es nicht RMS
- Overhead nicht stillschweigend ignorieren, wenn er die Annahmen verletzt

Notizen

Frage: Liegt $U = 85\%$ bei drei Tasks unter oder über $U_{LL}(3)$? Darüber — deshalb RTA, nicht automatisch Fail.

Response-Time Analysis (RTA)

Einfaches FP-Modell, $D_i \leq T_i$, $hp(i)$ = höherprioritäre Tasks:

$$R_i^{(0)} = C_i, \quad R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$$

- Aufrundung: Anzahl störender Jobs im Fenster
- Stopp bei Fixpunkt oder bei $R > D$; $R = D$ besteht
- Exakt nur in diesem Modell; kein stiller Blocking-/Jitter-Term

85%-Set, RMS-Prioritäten:

- $\tau_1: 1 \rightarrow 1, R_1 = 1 \leq 4$
- $\tau_2: 2 \rightarrow 3 \rightarrow 3, R_2 = 3 \leq 5$

Notizen

$hp(\tau_2) = \{\tau_1\}$. $2 + \lceil 2/4 \rceil \cdot 1 = 3$, dann stabil.

RTA für das 85%-Set

$\tau_3, C_3 = 2, hp = \{\tau_1, \tau_2\}$:

Schritt	Rechnung	Wert
0	C_3	2
1	$2 + \lceil 2/4 \rceil \cdot 1 + \lceil 2/5 \rceil \cdot 2$	5
2	$2 + \lceil 5/4 \rceil \cdot 1 + \lceil 5/5 \rceil \cdot 2$	6
3	$2 + \lceil 6/4 \rceil \cdot 1 + \lceil 6/5 \rceil \cdot 2$	8
4	$2 + \lceil 8/4 \rceil \cdot 1 + \lceil 8/5 \rceil \cdot 2$	8

- Fixpunkt $R_3 = 8 \leq 10$
- Kritischer Ablauf: 0–1 τ_1 , 1–3 τ_2 , 3–4 τ_3 , 4–5 τ_1 , 5–7 τ_2 , 7–8 τ_3
- 85 % liegen über $U_{LL}(3)$, dieses Set ist unter RMS dennoch nachgewiesen
- Daraus folgt nicht „85 % ist immer sicher“

Notizen

Bei Fenster 5: $2 + 2 + 2 = 6$. Bei Fenster 6: $2 + 2 + 4 = 8$. Releases bei $t = 4$ und $t = 5$. Quellen: Liu/Layland 1973; Audsley u. a. zur statischen Prioritätsanalyse.

Fazit: RMS in der Praxis

- Feste Prioritäten: geringer Laufzeitaufwand, Analyse vorab
- Überlast: eine für sich nachgewiesene höherpriori Teilmenge wird im unabhängigen präemptiven Modell nicht durch zusätzliche Low-Prio-CPU-Arbeit verdrängt
- Das ist keine Garantie „nur Low-Prio verpasst Fristen“
- Höchstpriori kann die eigene Frist verfehlen (Budget, Maskierung, Sperren)

Notizen

Gegenbeispiel gleich bei der Überlastfolie: $C = 5, T = D = 4$ scheitert allein.

Earliest Deadline First (EDF)

- Unter den bereiten Jobs: früheste absolute Deadline
- Ereignisgesteuert; geeignete Datenstruktur, nicht „jede ms komplett neu sortieren“ als Definition
- Nicht auf Soft Real-Time beschränkt
- Linux `SCHED_DEADLINE`: EDF **mit** CBS und Zulassungskontrolle — nicht ungeschütztes Lehrbuch-EDF

Notizen

Gleichstand heute: die **laufende** Aufgabe behält die CPU.

Die mathematische Eleganz von EDF

Für unabhängige, präemptive Tasks auf **einer** CPU mit $D_i = T_i$, ohne Overhead:

$$U \leq 1 \quad \text{genau dann schedulbar unter EDF}$$

- Gilt nicht für $D < T$ (40%-Gegenprobe)
- Gilt nicht automatisch auf dem H755 mit zweitem Kern oder unmodellierter Interferenz

Notizen

Notwendig und hinreichend nur in diesem Idealmodell.

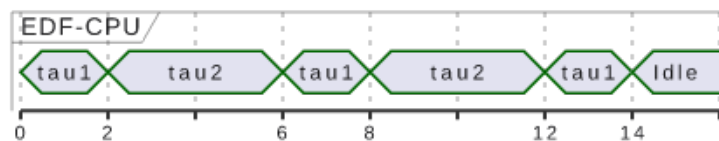
Visualisierung: Dynamisches EDF

97%-Set: $\tau_1 = (2, 5, 5)$, $\tau_2 = (4, 7, 7)$, synchron bei 0. $U = 2/5 + 4/7 = 34/35 \approx 97,14\%$.

RMS scheitert: 0–2 τ_1 , 2–5 τ_2 , 5–7 τ_1 ; bei $t = 7$ fehlt τ_2 noch 1 ms, D_2 verletzt. RTA: $4 \rightarrow 6 \rightarrow 8 > 7$.

EDF (Gleichstand: laufende Task bleibt):

- 0–2 τ_1/J_1 , 2–6 τ_2/J_1 , 6–8 τ_1/J_2 , 8–12 τ_2/J_2 , 12–14 τ_1/J_3
- Bei $t = 5$: Deadline 7 schlägt Deadline 10, τ_2 läuft weiter



Notizen

τ_1/J_2 ist 5–6 Ready und 6–8 Running. Frage: Warum unterbricht τ_1 bei $t = 5$ nicht?

EDF: Entscheidungen bis $t = 12$

t	Bereit, Regel
0	$d(\tau_1) = 5 < d(\tau_2) = 7 \rightarrow \tau_1$
2	τ_1 fertig $\rightarrow \tau_2$ ($d = 7$)
5	τ_2 läuft, $d = 7 < 10 \rightarrow$ bleibt
6	τ_2/J_1 fertig $\rightarrow \tau_1/J_2$ ($d = 10$)
7	τ_2/J_2 kommt, $d = 14 > 10 \rightarrow \tau_1$ bleibt
8	τ_1 fertig $\rightarrow \tau_2/J_2$
10	τ_1/J_3 , $d = 15 > 14 \rightarrow \tau_2$ bleibt
12	τ_2 fertig $\rightarrow \tau_1/J_3$ bis 14

Hyperperiode 35 (Notizen): 14–15 τ_2/J_3 , 15–17 τ_1/J_4 , 17–20 τ_2/J_3 , 20–22 τ_1/J_5 , 22–26 τ_2/J_4 , 26–28 τ_1/J_6 , 28–32 τ_2/J_5 , 32–34 τ_1/J_7 , 34–35 Idle.

Notizen

Bei gleicher absoluter Deadline bleibt die laufende Aufgabe (so bei $t = 30$ in der Hyperperiode). Simulation illustriert; der Nachweis für implizite Deadlines ist $U \leq 1$.

Der Haken an EDF: Überlast

- EDF ist deterministisch unter festen Regeln, nicht „chaotisch“
- Überlast kann weitere Misses verursachen — abhängig von Set und Miss-Behandlung
- Höchstpriorität H : tatsächlicher Bedarf 5 ms, $T = D = 4$ ms verfehlt die Frist **allein**
- Geplanten C und überschrittenen Bedarf getrennt benennen
- Strategien: verspäteten Job beenden, verwerfen oder Budget begrenzen — jede ändert das Verhalten
- Feste absolute Deadline-Marker bleiben; ein überfälliger Job wird nicht automatisch „wichtiger“ nur weil die Uhr läuft, solange seine absolute Deadline fest ist und die Regel „früheste d “ gilt

Notizen

Keine Empfehlung „Airbag = RMS“. Linux SCHED_DEADLINE schützt per CBS/Admission; das ist keine Garantie bei unzulässiger Gesamtlast. Einen Überlast-Gantt nur mit vollständigen Kosten zeichnen — hier reicht das Einzeljob-Gegenbeispiel.

Zusammenfassung: RMS vs. EDF

RMS (FP nach Rate)		EDF
Auswahl feste Priorität aus T		früheste absolute d
Test hier	LL hinreichend; RTA im FP-Modell	$U \leq 1$ bei $D = T$, eine CPU
75 % / 85 % / 97 %	75 % und 85 % halten; 97 % scheitert	97 % hält im Idealmodell
Überlast	Low-CPU verdrängt High nicht; eigene Budgets können trotzdem reißen	weitere Misses möglich; Strategie festlegen
Aufwand Priorität konfigurieren		Deadline-Ereignisse verwalten

Notizen

Keine universelle Branchenzuordnung. Beide brauchen das passende Fehlermodell.

Folie

52 / 66

Unter der Haube: Wie baut man einen Scheduler?

- Präemptiver RTOS-Port (Referenz FreeRTOS CM4F): SysTick und PendSV
- Labor heute: **kein** selbstgeschriebener Registerwechsel
- PendSV ist nicht „von Natur aus“ niedrigprior; der Port setzt die Priorität

Notizen

Öffentliche Referenz: FreeRTOS-Kernel `portable/GCC/ARM_CM4F/port.c`. Die lokal eingesetzte Version kann abweichen; `main` allein belegt sie nicht. Keine `.ioc` im Kursrepo.

Folie

53 / 66

Der Herzschlag: SysTick-Timer

- 24-Bit-Downcounter, Reload $LOAD = f_{Quelle} / f_{Tick} - 1$ (Wertebereich prüfen)
- Typische Tickquelle, nicht die einzige (anderer Timer, Tickless Idle möglich)
- Im üblichen CM4F-Port: SysTick **und** PendSV auf niedrigste IRQ-Priorität
- Kleinere NVIC-Zahl = höhere IRQ-Dringlichkeit; nicht mit der Taskpriorität verwechseln

Notizen

Keine unbelegte Behauptung „SysTick ist High-Prio“. Taktquelle am Projekt ablesen.

Code: Die SysTick-ISR

Konsistentes Muster: Zeit und Timeouts; Wechsel **anfordern**, Auswahl im PendSV ausführen. Eine lokale Variable `next` gilt nicht als übergebene Entscheidung, solange sie nicht gespeichert wird.

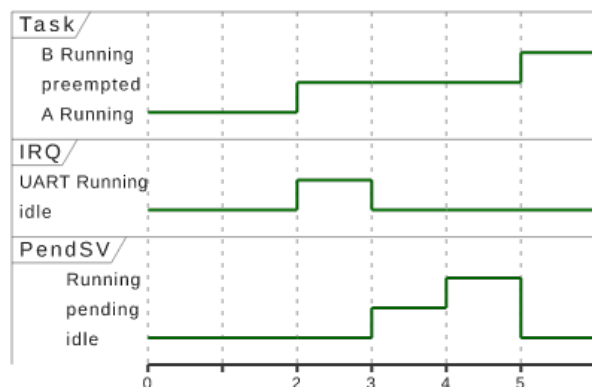
```
void SysTick_Handler(void) {
    os_tick_count++;
    os_update_timeouts();
    /* Wechsel nur vormerken, nicht die Register hier tauschen */
    os_request_switch(); /* setzt PendSV pending */
}
```

Notizen

Entweder nur Ready-Zustände hier aktualisieren oder die Auswahl zuverlässig hinterlegen. Kein unvollständiger Assembler als „der FreeRTOS-Port“.

Warum der Umweg über PendSV?

- Aufgeschobener Task-Wechsel: ISR setzt Pending, PendSV läuft wenn die Priorität drankommt
- Pending $\neq$ Running
- Eine UART-ISR verliert nicht automatisch Daten, weil SysTick eintritt
- Datenverlust: zu späte Bedienung, FIFO, zu lange Maskierung
- Höherpriorie ISR kann eine Task wecken; PendSV danach, vor dem nächsten Tick



Notizen

Verschachtelte Exceptions: höhere IRQ-Priorität vor PendSV. Maskierung verschiebt den Wechsel zusätzlich.

Folie

56 / 66

Der Task Control Block (TCB)

- RTOS-TCB: Stackpointer auf den gesicherten Kontext, Priorität, Zustand
- PC liegt im Frame, nicht als einzelnes TCB-Feld „neben“ dem Stack
- PSP: Taskkontext; MSP: Handlerkontext
- Labor-TCB: Funktionszeiger, Zustand, `wake_at` — **kein** Stackpointer

Notizen

Zwei TCBs nicht vermischen.

Folie

57 / 66

Der Software-Context-Switch (PendSV)

Basis-Hardwareframe: R0–R3, R12, LR, PC, xPSR (VL1).

Software zusätzlich, portabhängig: R4–R11, EXC_RETURN; FPU nur bei aktivem FP-Kontext.

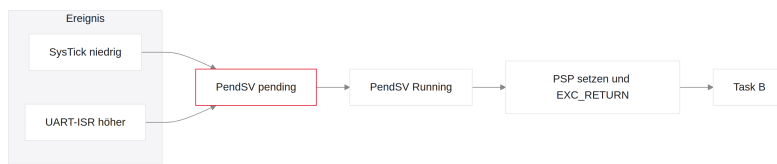
Restore, vollständig gedacht:

1. Kontextpointer der laufenden Task sichern
2. Ziel-TCB wählen
3. Ziel-SP laden, Software-Register zurück
4. **PSP auf den neuen Task-Stack setzen**
5. Exception Return über EXC_RETURN (BX LR nur mit diesem Wert)
 - Kein lauffähiger Minimal-Port in dieser Vorlesung; Referenz ist der eingesetzte FreeRTOS-Port

Notizen

Bisher fehlte das Setzen des PSP. Ohne EXC_RETURN ist der Rücksprung kein Task-Wechsel. PM0214.

Zusammenfassung: Die OS-Architektur



- Tick **oder** ISR kann den Wechsel vormerken

Notizen

Nicht mehr die schmale Kette „nur SysTick ist High und schaltet sofort“.

Vorbereitung für das Labor heute

- Board: NUCLEO-H755ZI-Q, MCU STM32H755ZIT6, Arbeit auf dem CM4
- Kein PendSV-Assembler
- Stacklos: `step()` kehrt zurück; Fortsetzung im TCB (`wake_at`, Zähler)
- Zeitbasis: z. B. `HAL_GetTick()`; der Tick prääemptiert die Steps nicht

Notizen

Ohne eigenen Stack kann ein normaler Aufruf keine Endlosschleife mitten im `yield` fortsetzen. Keine `.ioc` im Repo — Takt im Cube-Projekt ablesen.

Labor-Schritt 1: Die TCB-Liste

```
#define N 3
typedef enum { ST_READY, ST_BLOCKED } TaskState;
typedef struct {
    void (*step)(void); /* endliche Arbeit, kehrt zurück */
    TaskState state;
    uint32_t wake_at;    /* Tick; Wrap: (int32_t)(now-wake) >= 0
} MyTCB_t;
MyTCB_t task_list[N];
uint8_t rr_cursor;
uint8_t running_idx;    /* nur während step() gültig */
```

Notizen

Zustand nicht auf einem vermeintlich pausierten Stack.

Labor-Schritt 2: Die Scheduler-Logik

Zentraler Dispatcher, **kein** `step()` aus `step()`:

```
void dispatcher(void) {
    for (;;) {
        uint32_t now = HAL_GetTick();
        for (uint8_t i = 0; i < N; i++)
            if (task_list[i].state == ST_BLOCKED &&
                (int32_t)(now - task_list[i].wake_at) >= 0)
                task_list[i].state = ST_READY;
        int sel = -1;
        for (uint8_t k = 0; k < N; k++) {
            uint8_t i = (uint8_t)((rr_cursor + k) % N);
            if (task_list[i].state == ST_READY) { sel = i; break; }
        }
        if (sel < 0) { __WFI(); continue; } /* Wake-Quelle nötig */
        rr_cursor = (uint8_t)((sel + 1) % N);
        running_idx = (uint8_t)sel;
        task_list[sel].step(); /* kehrt zurück */
    }
}
```

- Höchstens N Kandidaten, dann Idle — keine Endlossuche
- `__WFI` nur mit aktiver Wake-Quelle (Tick)
- `os_delay`: `wake_at` setzen, Zustand Blocked, **sofort return** — kein stackhaltendes `os_delay` mitten in der Funktion

Notizen

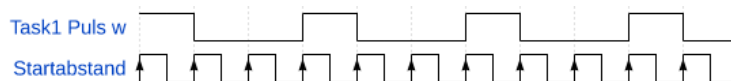
Geschachtelt wäre $A \rightarrow \text{yield} \rightarrow B \rightarrow \text{yield} \rightarrow C$ und der Stack wächst. Hier bleibt die Tiefe begrenzt.

Labor-Schritt 3: Hardware-Messung (Oszilloskop)

Pulsbreite $w_k = f_k - s_k$ (verstrichen, inkl. möglicher IRQs) und Startabstand $P_k = s_{k+1} - s_k$ unterscheiden.

Idealisierte Lehre, keine Boardmessung:

- $C_1 = C_2 = C_3 = 1$ ms: Task-1-Starts 0/3/6/9, Abstände konstant
- konstantes $C_2 = 3$ ms: Abstände konstant 5 ms — noch kein Jitter
- $C_2 = 1/3/1/3$, $C_1 = C_3 = 1$: Starts 0/3/8/11/16, Abstände 3/5/3/5, Spanne 2 ms; Pulsbreite von Task 1 bleibt 1 ms



- Gleiche Compiler-/Taktbedingungen, bekannter Pin, hohe Abtastrate, Marker für Task und ggf. ISR
- Zahlen erst nach Messung; Pin (z. B. PA0) nach Belegung prüfen

Notizen

„Präemptiv starr, kooperativ mit Jitter“ ersetzen: Jitter nur bei schwankender Last. Konstante Verlängerung ist ein Bias.

Labor-Schritt 4: Den Scheduler crashen

- Schritt ohne Rückkehr: weitere Callbacks bleiben aus; IRQs können weiterlaufen
- Separat, präemptiv: High ständig Ready $\Rightarrow$ Low verhungert; `yield` reicht nicht
- Deshalb Budgets, Blockieren und — ab Termin 3 — präemptives FreeRTOS mit Nachweis, nicht als Automatik

Notizen

Zwei Fehlerbilder nicht zu einem „Crash“ vermischen.

Zusammenfassung des Tages

- Ein Prozessor: $C, T, D, R \leq D$; 75 % / 85 % / 97 % durchgerechnet
- LL-Schranke hinreichend; 85 % via RTA ($R_3 = 8$); 97 % RMS fail, EDF hält im Idealmodell
- $U = 40\%$ mit $D = 3$ ms kann unmöglich sein
- SysTick/PendSV: beide im CM4F-Port niedrig; PSP setzen
- Labor: Dispatcher und endliche Steps

Notizen

Kontrollwerte: $U_{75} = 0,75$, $U_{85} = 0,85$, $U_{97} = 34/35$. RTA 75 %: 1, 3, 6. RTA 85 %: 1, 3, 8.

Ausblick auf Termin 3

- FreeRTOS auf dem Cortex-M4, präemptive Threads
- API: `osThreadNew`, `osDelay` — anderer Kontext als `step()`
- Prioritäten konfigurieren ist noch kein RMS-Beweis
- Vorbereitung: Funktions- und Strukturzeiger

Notizen

Mutex und Priority Inheritance folgen später, nicht heute.

Quellen

- Liu & Layland (1973), Scheduling Algorithms for Multiprogramming
- Audsley u. a., Response-Time Analysis für statische Prioritäten
- FreeRTOS-Kernel, Port GCC/ARM_CM4F (Version am Projekt prüfen)
- Linux-Doku `SCHED_DEADLINE` (EDF, CBS, Admission)
- ST PM0214, Cortex-M4 Exceptions / PSP / EXC_RETURN

Notizen

Keine Branchenstatistiken. Keine Messwerte ohne Oszilloskop-Protokoll.